

## Modul 04: MediaTypes, Mappers und Routing

**Ziel:** Wir nutzen die Asp.Net Features für Controllers aus und berücksichtigen Formatters, MIME Types und Datenmodellierung.

**Tools:** Visual Studio 2022

**Dauer:** ~30min

### 1 Movie-Controller verbessern

Man könnte im Movie-Controller die RückgabeTypen, IActionResult und ActionResult verwenden. Genauso kann man auch bei den Validierungen mit den http-StatusCodes arbeiten.

```
[Route("api/[controller]")]
[ApiController]
public class MoviesController : ControllerBase
{
    private readonly IMovieService _service;

    public MoviesController(IMovieService service)
        => _service = service;

    [HttpGet]
    public async Task<ActionResult<IEnumerable<Movie>>> GetMovies()
    {
        return await _service.GetMovies();
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<Movie>> GetMovie(int id)
    {
        var movie = await _service.GetMovie(id);

        if (movie == null)
        {
            return NotFound();
        }

        return movie;
    }

    // usw.
}
```

## 2 Konfiguration der Controller in der Program.cs

Referenzieren Sie die Klassenbibliothek MovieStore als Projektabhängigkeit und binden Sie den MovieService in der Program.cs als Singleton ein:

```
builder.Services.AddSingleton<IMovieService, MovieService>();
builder.Services.AddControllers().AddJsonOptions(options =>
{
    // Enums in Datenmodellen als Text statt Integers ausgeben
    // indem wir diese JsonSerializer Options konfigurieren
    options.JsonSerializerOptions.Converters.Add(
        new JsonStringEnumConverter()
    );

    // XML als Response Type ermöglichen
}).AddXmlSerializerFormatters();
```

## 3 Data Transfer Objects, Mappers und Validierung

Legen Sie eine Datenklasse MovieResultDto an, welches das Domänen-Modell transformiert. Eine weitere Klasse ClientMovieDto soll Validation Attribute enthalten, die im Controller mittels dem ModelState validiert werden.

Eine statische Klasse soll Erweiterungsmethoden .ToDomainModel() und .ToDto() enthalten, welche die Konvertierung der Datenmodelle ermöglicht.

## 4 Web API testen

Starten Sie die WebAPI-App erneut und testen Sie die bestehenden sowohl als auch neuen Funktionalitäten. Sie können zum Testen Swagger, <http-files> oder ein anderes Werkzeug Ihrer Wahl verwenden.

## 5 Bonusaufgaben

Erstellen Sie eine Klasse SupportedImageAttribute, welches den Url-Pfad "ImageUrl" auf unterstützte Bildformate wie jpeg oder png prüft.

Legen Sie eine Klasse SearchParams mit den Eigenschaften PublishDate, Genre und PriceRange an. Diese werden via SearchQuery aufgelöst. Implementieren Sie eine Filter-Funktion für die Filme.